Chris Toomey, Jenice An, and Tongyu Guo
ADAN 8888 - Applied Analytics Project - Spring 2026
Module 8 Assignment - Develop Third Modeling Approach

HelpHerInvest: A machine learning-based investment recommendation system that aligns user interests and risk preferences with benchmark stock evaluation.

Modeling Approach: LLM

The modeling approach for this week was an LLM-based file reasoning workflow. The model was given two uploaded files: a stock symbol reference dataset and a company/fundamental dataset. It then had to interpret a user's industry or investing theme, select exactly 20 relevant stocks, rank them using the available company data, explain the basis for the ranking, and clearly state data limitations.

This approach made sense because the task was not only about ranking stocks, but also about understanding natural language, retrieving relevant symbols, combining information across multiple files, and communicating limitations clearly. The earlier notebook already showed that the workflow relied on OpenAI file inputs and that the available data was not sufficient for a strong traditional supervised learning model, since many standard predictor columns were missing

Complexity of the Modeling Approach

The modeling approach was moderately complex. It was more involved than basic prompting because it required file upload and retrieval, prompt engineering, automatic parsing of outputs, metric calculation for both training and validation prompt sets, and final model selection based on validation performance.

At the same time, it differs from a traditional machine learning workflow because the notebook does not train model weights locally. Instead, the "model tuning" focuses on LLM configuration choices such as model selection, prompt structure, and output token budget. This still represents a meaningful level of model complexity because output quality depends heavily on whether the model could follow structured instructions, ground its reasoning in the uploaded files, and acknowledge missing data.

Hyperparameters

We evaluated three variations of the LLM setup:

Variation 1: V1_baseline_mini

Model: gpt-5-mini

Max output tokens: 900

Prompt style: baseline prompt

This version used a simpler developer prompt similar to the original notebook setup.

Variation 2: V2_strict_mini

Model: gpt-5-mini

Max output tokens: 1100

Prompt style: strict structured prompt

This version used the same smaller model as Variation 1 but enforced a more structured response format. The goal was to test whether prompt structure alone could improve performance.

Variation 3: V3_strict_gpt54

Model: gpt-5.4

Max output tokens: 1100

Prompt style: strict structured prompt

This version kept the structured prompt from Variation 2 but used a stronger model. The goal was to test whether combining a stronger model with a structured prompt would improve performance.

These configurations were chosen to evaluate three key factors: model choice, prompt structure, and output length.

Training and Validation Datasets

Because this was an LLM evaluation workflow rather than a traditional trainable ML model, we defined training prompts and validation prompts.

The training prompt set contained six prompts:

- I like the fashion industry.
- I am interested in renewable energy.
- I want healthcare stocks.
- I like semiconductor companies.

- I am interested in cybersecurity.
- I want consumer staples stocks.

The validation prompt set contained four held-out prompts:

- I want travel and leisure stocks.
- I am interested in electric vehicle companies.
- I like biotech.
- I want cloud computing stocks.

The training prompts were used to compare and tune the variations, while the validation prompts were used to select the best model for the week.

Model Performance Metrics

We used the following evaluation metrics:

1. **selected_20_score**
   This metric measured whether the model returned exactly 20 selected stocks. This metric made sense because the task explicitly required the model to identify 20 relevant symbols.
2. **ranking_20_score**
   This metric measured whether the model returned a complete ranking of 20 stocks. This was important because the assignment required both selection and ranking.
3. **has_ranking_basis**
   This metric measured whether the model explained the reasoning behind the ranking. This mattered because a ranking without justification would not be interpretable.
4. **has_data_limitations**
   This metric measured whether the model explicitly discussed data limitations. This was especially important given the earlier finding that many predictor fields in the company dataset were sparse or missing.
5. **grounding_score**
   This metric measured whether the model used grounded language indicating awareness of missing, sparse, or insufficient data. This metric made sense because the previous notebook output showed that grounded discussion of limitations was essential to using the dataset responsibly.

6. **overall_score**
   This metric was the average of the five task-quality metrics above. It provided one summary measure of how well each variation completed the full task.
7. **latency_seconds**
   This metric measured how long the model took to produce an answer. Latency was included because model quality and efficiency are both relevant in comparing configurations.

Validation Metric Values for all 3 Variations

Variation	Model	Dataset	selected_20_score	ranking_20_score	has_ranking_basis	has_data_limitations	grounding_score	overall_score	latency_seconds
V1_baseline_mini	gpt-5-mini	Train	0	0	0	0	0	0	22.56
V1_baseline_mini	gpt-5-mini	Validation	0	0	0	0	0	0	18.392
V2_strict_mini	gpt-5-mini	Train	0	0	0	0	0	0	23.615
V2_strict_mini	gpt-5-mini	Validation	0	0	0	0	0	0	20.685
V3_strict_gpt54	gpt-5.4	Train	1	1	1	1	1	1	28.077
V3_strict_gpt54	gpt-5.4	Validation	1	1	1	1	1	1	26.565

Variation	Model	Dataset	selected_20_score	ranking_20_score	has_ranking_basis	has_data_limitations	grounding_score	overall_score	latency_seconds
V1_baseline_mini	gpt-5-mini	Train	0.0	0.0	0.0	0.0	0.0	0.0	22.560
V1_baseline_mini	gpt-5-mini	Validation	0.0	0.0	0.0	0.0	0.0	0.0	18.392
V2_strict_mini	gpt-5-mini	Train	0.0	0.0	0.0	0.0	0.0	0.0	23.615
V2_strict_mini	gpt-5-mini	Validation	0.0	0.0	0.0	0.0	0.0	0.0	20.685

V3_strict_gpt54	gpt-5.4	Train	1.0	1.0	1.0	1.0	1.0	1.0	28.077
V3_strict_gpt54	gpt-5.4	Validation	1.0	1.0	1.0	1.0	1.0	1.0	26.565

The evaluation results show a major difference between the first two variations and the third variation.

Analysis of Results

The results show a clear difference between the first two variations and the third variation.

Variation 1: V1_baseline_mini

Variation 1 had an overall score of 0.0 on both the training and validation datasets, with 0.0 across all individual metrics. This suggests that the baseline prompt paired with the smaller gpt-5-mini model did not produce outputs in a form that satisfied the scoring framework. In practice, the model was not consistently returning the sections needed for evaluation.

Variation 2: V2_strict_mini

Variation 2 also resulted in an overall score of 0.0 for both training and validation. This indicates that simply tightening the prompt structure alone was not enough when using the gpt-5-mini model. Even with clearer and more constrained instructions, the smaller model was still unable to reliably produce the expected 20-symbol selection, full ranking, ranking basis, and data limitations sections.

Variation 3: V3_strict_gpt54

Variation 3 achieved an overall score of 1.0 on both training and validation, and it scored 1.0 on every metric. This shows that the combination of a stronger gpt-5.4 model, a structured prompt, and a higher token limit allowed the system to consistently return complete and well-structured outputs, including the required 20-symbol selection, ranking, reasoning, and discussion of limitations.

Overall, performance did not improve gradually across variations. Instead, there was a clear jump between the smaller gpt-5-mini configurations and the stronger gpt-5.4 configuration. This suggests that model capability had a larger impact than prompt structure alone. The stricter

prompt was necessary for consistency, but the stronger model made it possible to follow the format and produce fully scorable responses.

Winning Model Variation for Week 8

The best model for the week was **V3_strict_gpt54.**

This model achieved the highest validation performance across all evaluation metrics:

selected_20_score = 1.0

ranking_20_score = 1.0

has_ranking_basis = 1.0

has_data_limitations = 1.0

grounding_score = 1.0

overall_score = 1.0

It was the only model that consistently produced complete, structured, and grounded outputs for both training and validation prompts. Although it had the highest latency, the assignment prioritized validation performance, and this model outperformed the other configurations.